

Ishan Suryawanshi

Pune, Maharashtra | ishans7325@gmail.com | github.com/RedPaladin7

ABOUT

I am a Data Science Engineering student with a strong bias toward depth over breadth. My technical background bridges advanced mathematics, cryptography, and rigorous systems programming. Rather than just making components talk to each other, I focus on mastering the underlying mechanics—from complex quantitative frameworks to secure networks and highly concurrent environments. This instinct for building foundational infrastructure correctly shapes how I approach complex, large-scale engineering challenges.

EDUCATION

Manipal Institute of Technology, Karnataka *Manipal, Karnataka*
Bachelor of Technology in Computer Science & Data Science *Expected May 2027*
CGPA: 8.62/10.00 | Relevant Coursework: Algorithms, Probability & Statistics, Numerical Methods, Distributed Systems, Machine Learning, Database Systems

TECHNICAL SKILLS

Languages: Python, C++, Go, Solidity, Java, JavaScript/TypeScript, SQL

Libraries & Frameworks: NumPy, SciPy, Streamlit, Plotly, PyTorch, TensorFlow, Hugging Face Transformers, libp2p, GossipSub, Hardhat/Foundry, Linux epoll, CMake, Apache Spark, Hadoop/HDFS, Prometheus, Big Query Logging, React, Spring Boot, MCP

Concepts: Quantitative Finance, Derivatives Pricing, Machine Learning, Deep Learning, Transformer & Sequence Models, Big Data Processing, Zero-Knowledge Proofs, P2P Networking, Smart Contracts, Monte Carlo Simulation, Low-Latency Systems, Event-Driven Architecture, Non-blocking I/O, Load Balancing, Lock-Free Concurrency, Data Structures & Algorithms, Operating Systems, Object-Oriented Programming (OOP), DBMS, Systems Design

WORK EXPERIENCE

Goldman Sachs *May 25 – July 17, 2026*
Summer Engineering Analyst Intern, Asset Management – Middle Office Technology

- Designed and built an end-to-end SRE observability dashboard for a large, multi-repo trading application, consolidating health data across the four core SRE pillars (availability, latency, capacity, throughput) that was previously scattered across a time-series metrics store, an application logging system, and a manual trade-tracing tool into a single interface for engineers and managers.
- Built an automated SLO monitoring page and a trade-tracing visualization tool that combined live metrics with error logs to surface real-time service health and error budgets, and mapped complex, non-linear transaction flows into interactive diagrams showing expected vs. actual process paths for any given trade ID — replacing manual, query-by-query investigation, in close collaboration with senior engineers to reverse-engineer undocumented system behavior.
- Built a natural-language chatbot (using MCP) enabling non-technical stakeholders to query system health and trade status conversationally, correlating data across metrics and logging systems to distinguish handled errors from actual service failures; researched and scoped an evaluation framework — including a golden-dataset approach for benchmarking prompt, tool, and model configurations — to quantify and improve response accuracy ahead of productionization.
- Adopted a disciplined AI-assisted development workflow on a large, unfamiliar codebase — scoping goals, breaking work into incremental plan files, and validating each change against tests — to reliably ship correct changes rather than relying on unguided AI output.

PROJECTS

L4 TCP Load Balancer 2026
C++23, Linux epoll, timerfd, CMake

- Built a single-threaded, zero-dependency L4 TCP reverse proxy on Linux epoll — one reactor multiplexing accept, bidirectional splice, timerfd health probes, and a live /stats endpoint — with round-robin and least-connections routing, automatic failover, and AddressSanitizer-gated fd/lifetime correctness.

- Engineered a mixed-readiness epoll reactor — edge-triggered drain-until-EAGAIN on connection sockets; level-triggered listen (64-accept cap, EMFILE pause) and timerfd/signalfd — with RAII file descriptors, deferred reclaim after each epoll_wait batch, and std::expected on the I/O path.
- Contained slow-peer memory growth with per-direction buffers and hysteresis watermarks (256 KiB / 64 KiB) that pause EPOLLIN under backpressure and speculative-drain on resume — holding RSS flat under a 10 MiB slow reader without stalling the reactor.
- Implemented round-robin and least-connections over a health-aware pool — timerfd probes in the same loop (TCP connect / optional HTTP GET, 3-down/2-up thresholds) — so a killed backend leaves rotation without a second thread or mutex and rejoins automatically; fail-closed when nothing is healthy.
- Exposed live per-backend active/total/health counters on a dedicated /stats port used as the failover assertion surface; under Release load (one pinned core) recorded ~281 rps on 1 MiB bulk (C=1), 100 MiB curl matching direct at 0.26 s, and an exact 1/3 round-robin split with rejected=0.

Decentralized P2P Texas Hold'em Poker Engine 2026
Go, libp2p

- Built a serverless multiplayer Texas Hold'em engine combining cryptographic mental-poker dealing, a replicated Hold'em state machine (blinds, side pots, 7-card evaluation, multi-hand sessions), and a libp2p mesh — with no central game server and no trusted dealer.
- Implemented 2048-bit SRA commutative-encryption dealing — seat-order encrypt-then-permute shuffle, layered partial decrypts, and non-interactive Chaum-Pedersen zero-knowledge proofs — cryptographically hiding opponents' hole cards until showdown.
- Built a libp2p networking layer with GossipSub broadcasting, Noise transport, Ed25519-signed envelopes, per-sender replay protection, and mDNS discovery, plus an application-level action sequencer that produces identical pots and winners on every honest replica.
- Designed a fault-tolerance protocol using heartbeats, 2/3-majority timeout votes, and Shamir secret sharing of each peer's private SRA exponent — force-folding a silent seat and recovering the missing key so the table can finish remaining peels.
- Designed a replicated state machine using authenticated total order, a socket-free Hold'em reducer (no local deck), and signed-log equivocation detection — catching conflicting payloads at one seq after the fact without a BFT quorum on every fold.

LANGUAGES

English: Fluent | Hindi: Fluent | French: Beginner